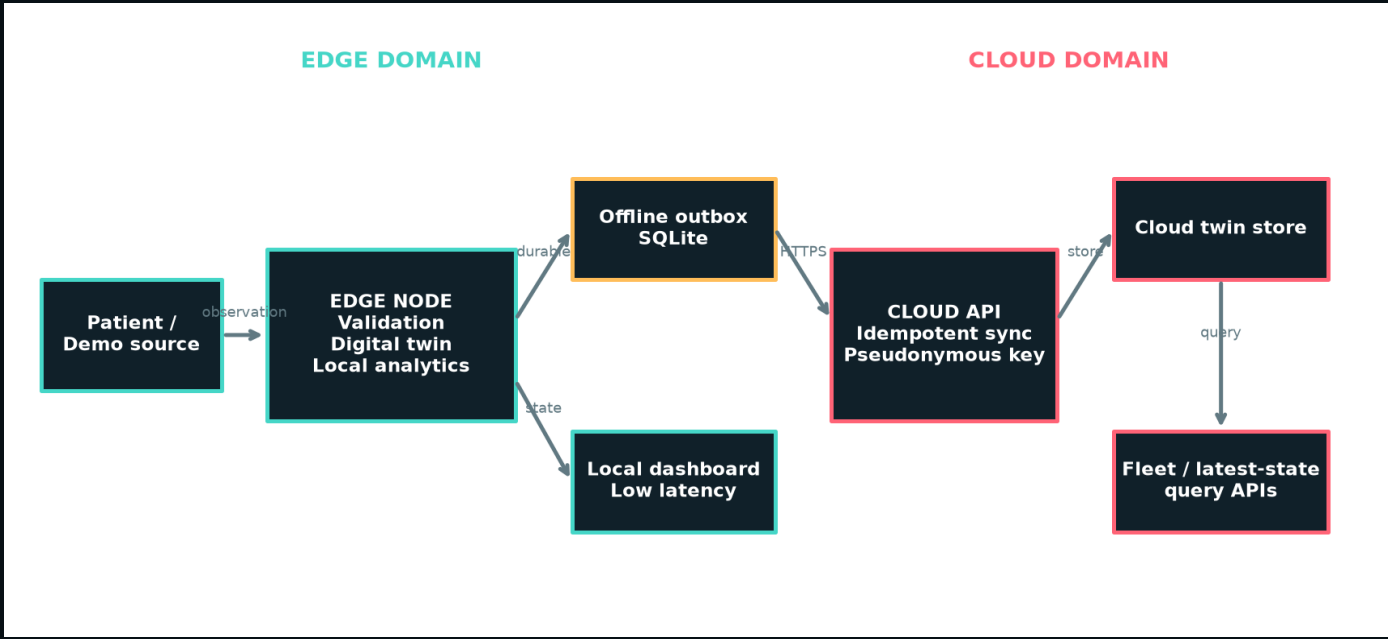
# CRITICALCARE-TWIN

Cloud-Edge Patient Digital Twin Architecture




Detailed technical architecture, real dataset evidence, test results, deployment model, and scientific limitations

Research and educational software prototype - not clinically validated

# 1. Executive summary

CRITICALCARE-TWIN is a patient-specific digital twin research prototype split across an edge domain and a cloud domain. The edge node accepts timestamped observations, validates them, updates the persistent physiological state, runs low-latency analytics, and records an auditable snapshot. A durable outbox then transfers a deliberately reduced, pseudonymous state summary to the cloud service. The cloud stores versioned summaries and exposes latest-state and fleet-level query endpoints.

This separation is intentional. Immediate state synchronization does not depend on network availability. Cloud connectivity adds longitudinal retention and multi-edge aggregation, but a network outage cannot stop the local twin. In the current free local demonstration, the cloud boundary is executed as a second FastAPI process. The API contract is deployable to a cloud host without changing edge behavior; this document does not claim that a commercial cloud account is configured.

## What changed

- Added a deployable FastAPI cloud service with health, sync, latest-state, and fleet-summary endpoints.

- Added an offline-safe SQLite edge outbox and explicit synchronization audit.

- Pseudonymized the source patient identifier before any cloud payload is built.

- Integrated cloud synchronization into every real twin snapshot while preventing scenario clones from publishing.

- Consolidated eight dashboard pages into seven by merging data provenance and model evaluation into Evidence Lab.

# 2. System context and trust boundaries

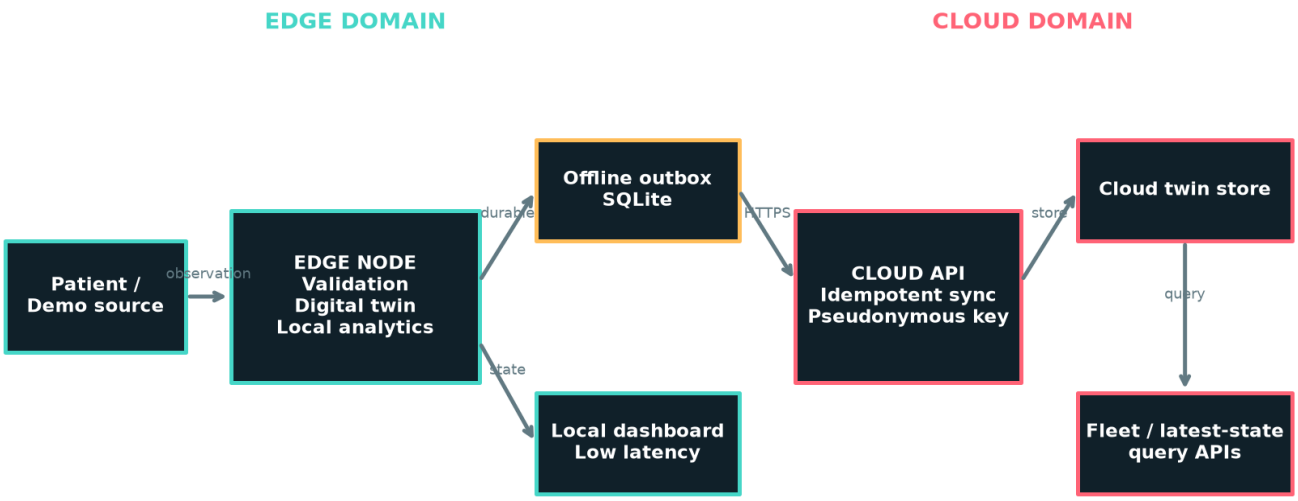



Figure 1. The edge domain owns raw observations, direct patient identifiers, state estimation, and low-latency behavior. The cloud domain receives only a pseudonymous patient key, twin ID, version, state vector, derived values, risk output, uncertainty, provenance class, and timestamps. The local demonstration uses HTTP on loopback; a real deployment must terminate TLS and authenticate edge sites.

## Edge responsibilities

- Ingest and validate observations against configured physiological bounds.

- Maintain X(t-1), X(t), delta X, rolling state history, baseline B, and uncertainty.

- Run model inference, anomaly scoring, and short-horizon forecasting locally.

- Persist patient, observation, event, risk, and snapshot records in SQLite.

- Queue cloud envelopes when the service is unavailable and retry in order.

### Cloud responsibilities

- Accept idempotent messages keyed by message_id and preserve twin version.

- Store pseudonymous longitudinal state summaries across sessions and edge sites.

- Expose the latest state for one pseudonymous key and aggregate fleet counts.

- Remain independent of the Streamlit presentation layer.

## 3. End-to-end lifecycle

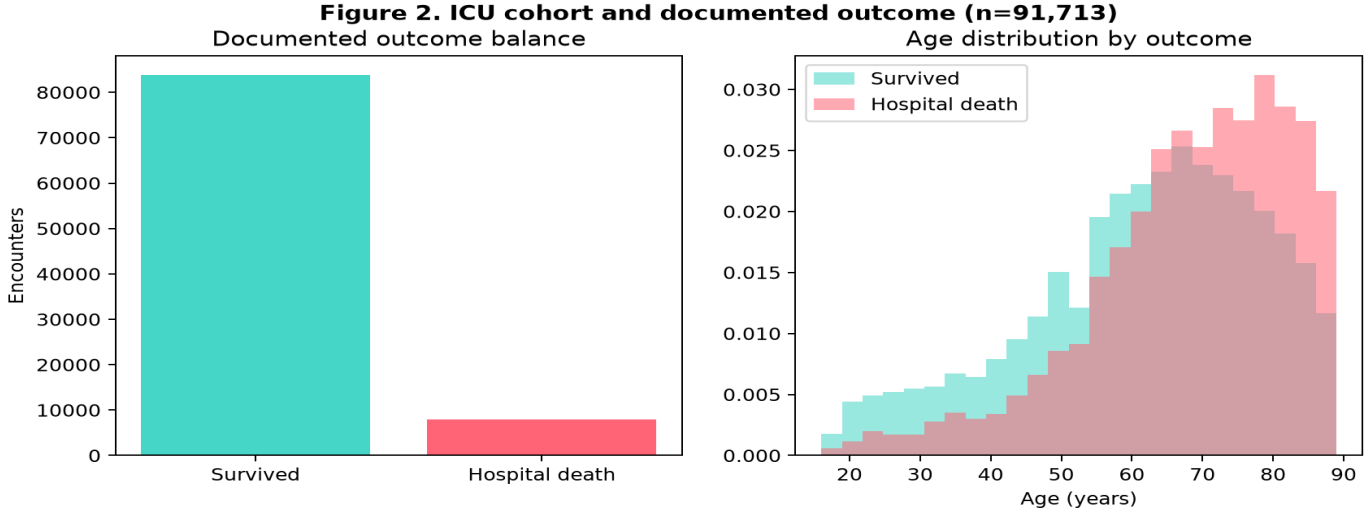
At time t the source produces observation Y_t. The edge validates Y_t, then computes X_t = F(X_(t-1), Y_t, q_t), where q_t is measurement quality. Derived features Z_t = G(X_t, B, history) feed the supervised hospital-death model, the independent anomaly detector, and the forecast. A snapshot is written locally before a cloud envelope is attempted. If the API accepts it, an audit row records success. If not, the unchanged JSON envelope is placed in the durable outbox.

| Step | Component | Output |
|---|---|---|
| 1 | Telemetry / source adapter | Timestamped Observation |
| 2 | Edge validation | Range/type decision |
| 3 | State estimator | Filtered X_t and delta X |
| 4 | Feature engine | Derived Z_t |
| 5 | Analytics | Risk, anomaly, forecast |
| 6 | Local repository | Durable snapshot and events |
| 7 | Edge gateway | Pseudonymous cloud envelope |
| 8 | Cloud API | Accepted versioned state |
| 9 | Cloud store | Latest-state and fleet queries |

# 4. Dataset evidence

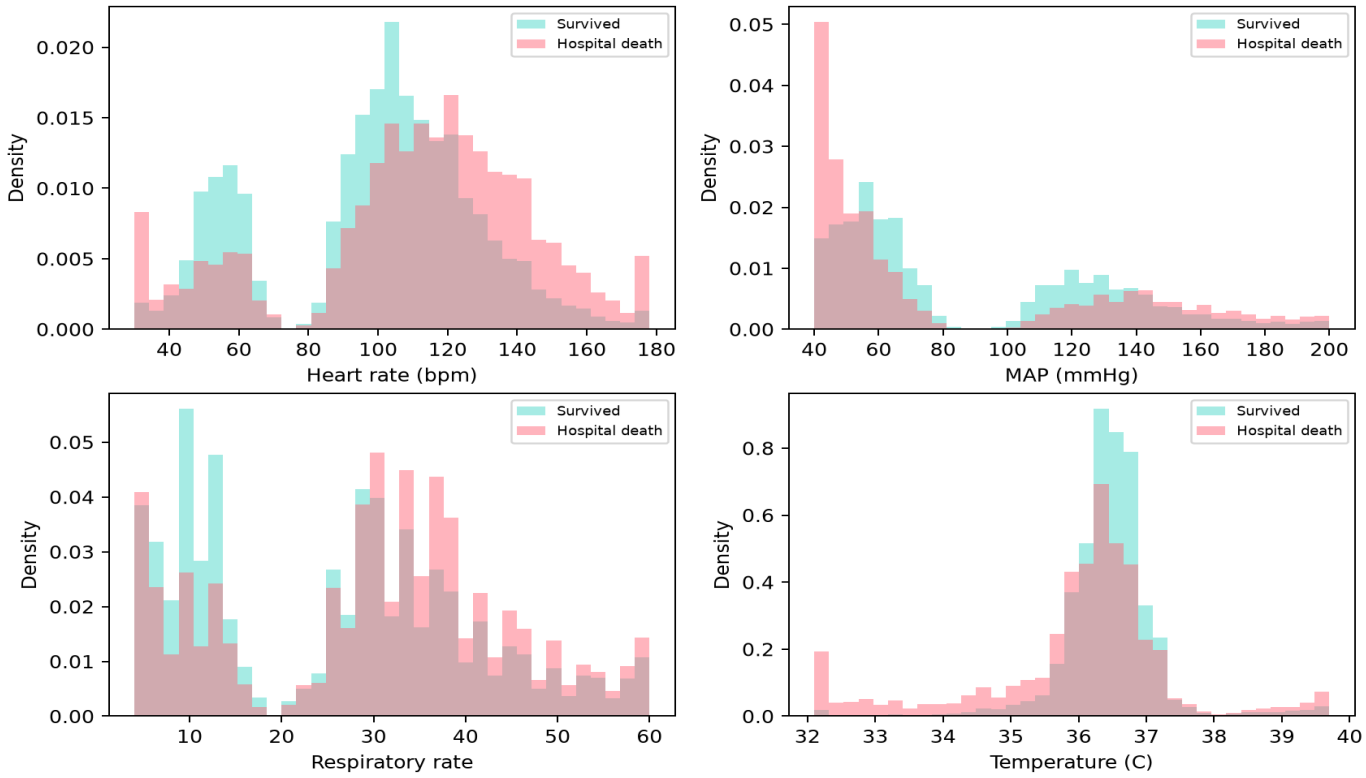
The primary source is the downloaded ICU encounter table from mitishaagarwal/patient. It contains 91,713 rows and 85 columns. It is cross-sectional at encounter level: first-hour and first-day minima/maxima are summaries, not an ordered bedside time series. The other downloaded datasets are kept separate because no scientifically defensible common patient identifier exists.

**Figure 2. ICU cohort and documented outcome (n=91,713)**

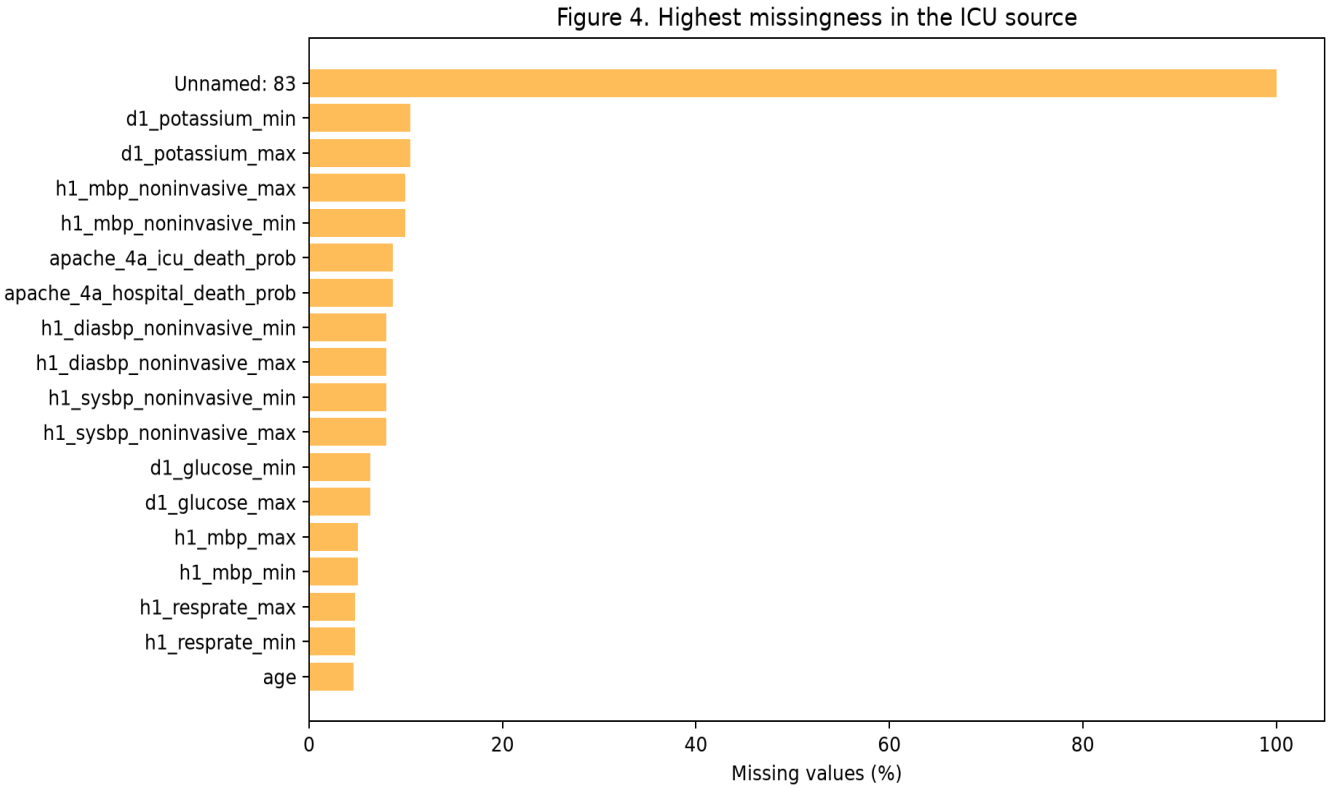



The documented hospital_death positive rate is 8.63%. Class imbalance motivates PR-AUC alongside ROC-AUC.

**Figure 3. Observed ICU admission physiology by outcome**

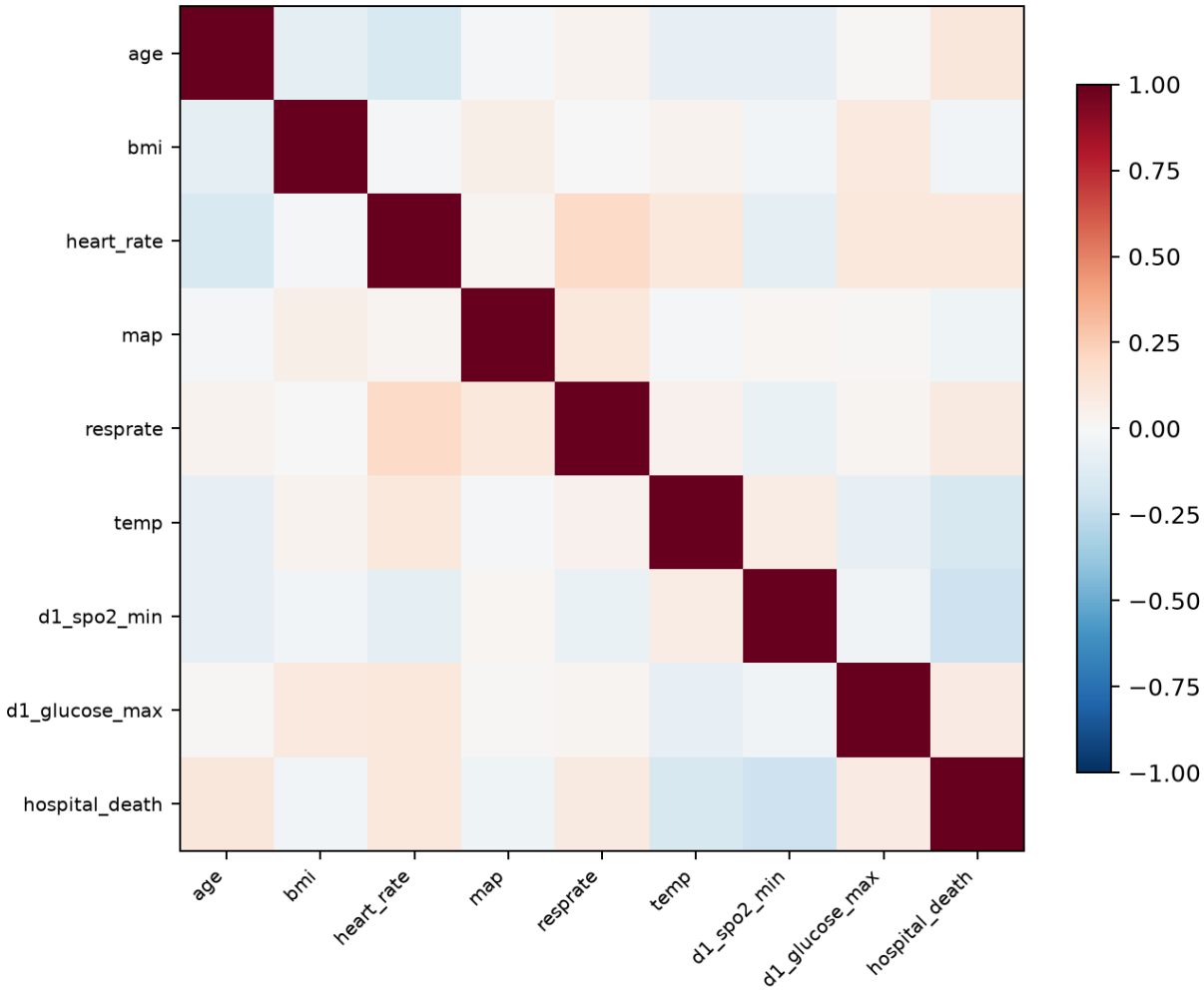



Distributions are descriptive views of observed source columns. They do not establish clinical thresholds or causal relationships.




Missingness is preserved during preprocessing and median-imputed inside the model pipeline. Twin uncertainty separately reflects missing observations.

Figure 5. Pairwise correlations (descriptive, not causal)

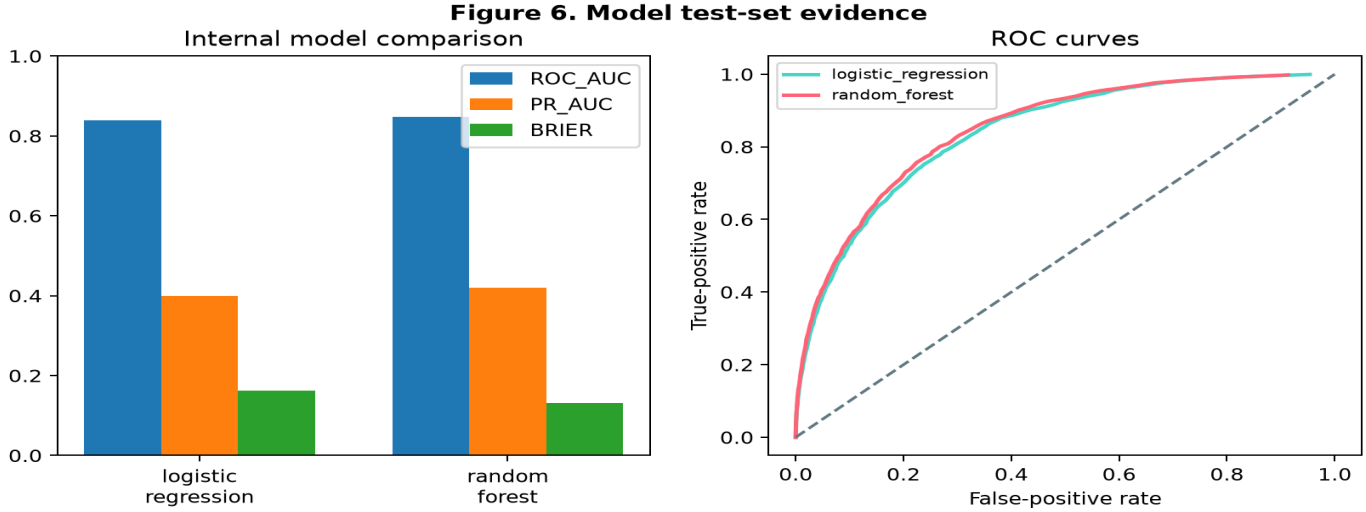



Correlations are descriptive and may reflect treatment, selection, site, and coding effects.

# 5. Model and test-set evidence

The downstream supervised task predicts the source field hospital_death. Logistic regression and a class-balanced random forest use a fixed 80/20 stratified encounter split. APACHE mortality probabilities, patient/encounter identifiers, and target-adjacent fields are excluded. The selected random forest reached ROC-AUC 0.849 and PR-AUC 0.420; these are internal metrics only.

**Figure 6. Model test-set evidence**




## Interpretation limits

- Random splitting does not establish future-site or temporal generalization.

- Feature importance and displayed contributions are associations, not causal effects.

- The model output is hospital-death probability for the dataset task, not a clinical diagnosis.

- Replay telemetry is simulated and is never used to fabricate training labels.

# 6. Cloud API and edge synchronization contract

| Endpoint | Purpose |
|---|---|
| GET /health | Service mode and stored snapshot count |
| POST /v1/twins/sync | Idempotently accept one CloudTwinEnvelope |
| GET /v1/twins/{patient_key}/latest | Return latest version for a pseudonymous patient |
| GET /v1/fleet/summary | Count snapshots, pseudonymous patients, and sites |

The edge derives patient_key as the first 20 hexadecimal characters of SHA-256(salt:patient_id). This is pseudonymization, not anonymization. Production use would require managed secrets, key rotation, authenticated transport, access control, audit retention, and formal privacy review. Scenario clones explicitly remove their edge gateway so experimental branches cannot enter the cloud record.

# 7. Persistence and failure behavior

- Local twin SQLite: patients, twins, observations, snapshots, events, risk predictions, simulations, model metadata.

- Edge outbox SQLite: unsent immutable envelopes plus successful-sync audit.

- Cloud SQLite demo: idempotent messages indexed by patient_key and observed time.

- Offline behavior: local synchronization continues; outbound messages queue; flush retries in chronological order.

- Duplicate behavior: cloud INSERT OR IGNORE on message_id makes retransmission idempotent.

# 8. Dashboard simplification

The former Twin Architecture page is replaced by Cloud & Edge, which combines live service health, queue depth, sync count, deployment boundary, and architecture. Data & Provenance and Model Lab are merged into Evidence Lab tabs. This removes redundant navigation while preserving scientifically important evidence. Command Center, Live Twin, State Explorer, Risk & Explainability, What-If Lab, Cloud & Edge, and Evidence Lab remain.

# 9. Verification

- Unit tests cover initialization, validation, state transitions, history, event generation, features, persistence, and scenario isolation.

- Cloud tests verify sync acceptance, latest-state retrieval, fleet aggregation, offline buffering, and absence of raw patient IDs in outbound JSON.

- Dashboard AppTest renders all seven pages and runs a what-if scenario without mutating the actual twin.

- The full stack is smoke-tested with the FastAPI service on port 8000 and Streamlit on port 8501.

# 10. Deployment guide

Development uses two processes so edge and cloud failure modes are observable. Start the cloud boundary with python scripts/run_cloud.py, then start the edge/dashboard with python -m streamlit run app.py. The edge target defaults to http://127.0.0.1:8000. A deployment can replace this URL with a TLS endpoint while retaining the same envelope and retry behavior.

## Production hardening required

- Mutual TLS or signed service credentials per edge site.

- Secret-managed HMAC or tokenization instead of a code-default salt.

- Managed database with encryption, backups, retention policy, and row-level access control.

- Schema versioning, dead-letter queues, observability, rate limits, and disaster recovery.

- Clinical governance, external validation, safety case, and regulatory assessment.

# 11. Scientific and ethical limitations

This system is a research and educational prototype. It is not a medical device and has no clinical validation. The cloud service does not change that status. Synthetic replay is a demonstrative time-indexed stream around an observed baseline, not real telemetry. The anomaly score, instability metric, forecast, and uncertainty layer are prototype methods. No output should diagnose, prescribe, triage, or manage a patient.

# 12. Traceability

| Concern | Implementation |
|---|---|
| Twin state | src/twin/state.py, patient_twin.py |
| Synchronization | src/twin/synchronizer.py |
| Edge outbox | src/edge/gateway.py |
| Cloud API | src/cloud/service.py |
| Persistence | src/twin/repository.py |
| Dashboard | app.py, src/ui |
| Dataset audit | reports/DATASET_AUDIT.md |
| Model evidence | reports/MODEL_EVALUATION.md |
| Tests | tests/ |